Using Generative AI To Generate a Policy for controlling a virtual ADROIT Hand from Human Examples

Gene Foxwell

Woolf University / Udacity School of Artificial Intelligence

Generative AI Applications

Overview

This project looks at applying Generative AI Techniques via the Transformer architecture to generate trajectories to solve the ADROIT Hand Relocate Reinforcement Learning problem. We will approach this problem using the human generated trajectories in D4RL dataset. We'll explain this dataset how our model design was done, as well as our training approach. We will also cover the various methods we employed to work around the limited number of trajectories available in the dataset. Finally, we'll discuss ethical / bias considerations for this model and future steps for improvement.

Figure 1 Hand in Action

Dataset Description

We approached this problem using human data generated (Fu 2020) from the ADROIT Relocate problem (Rajeswaran 2017). In the ADROIT Relocate problem, a 24 DoF robotic hand is tasked with picking up a small ball off a table and placing it down at a target location. We control the hand by providing a sequence of joint angles for each of the actuators as well as

positional / rotational angles for the hand's overall location in the world. For this problem, the D4RL group collected 25 human generated trajectories using a virtual environment combined with a CyberGlove[1] apparatus for input.

Each trajectory in the dataset has three key pieces of information:

Actions:

As mentioned earlier, actions represent joint angles for the ADROIT Hand. Our system will learn to generate sequences of actions using a Transformer model. Actions are represented as a 30-dimensional vector space. These represent the 24 DoF for the hands – 4 DoF for the First, Middle, and Ring Fings, 5 DoF for the Little Finger, Thumb, 2 DoF for the write, 3 values for the hand's global location (centered at the palm) and 3 values for the hand's global rotation. All inputs are clipped between [-1, 1].

Observations:

Observations are represented as 39-dimensional vectors. This observation space contains information about the hands current state (the same states used to represent the hand for actions). In addition, it has 3 entries representing the difference in position between the palm of the hand and the target, 3 entries representing the difference in position between the palm of the hand and the target, and 3 entries representing the difference in position between the ball and the target.

Reward:

- Rewards offered by the Human generated dataset are heavily shaped. At each step, the environment provides the following rewards:

[1] CyberGlove III CyberGlove III — CyberGlove Systems LLC

- Increase negative reward the further the hand is from the ball.
- A reward of 1 if the ball is off the table.
- A negative reward representing how far the ball is from the target when the ball is in hand.
- A reward of 10 when the ball is close to the target (less than 0.1 meters)

Each trajectory had between [297, 527] steps, for a total of 9942 total recorded (observation, action, reward) triplets recorded from the humans' interactions with the environment.

Before training our Transformer, we preprocessed the dataset. Our preprocessing involved the following steps:

- Created a timestamp array for each trajectory.
- Calculated "returns_to_go" – the cumulative reward for each timestep of a trajectory.
- Scaled "returns_to_go" by 3600 – the average reward humans received when performing this task using the virtual environment.

Model Design and Training Approach

For this task we based our model design on the Decision Transformer approach (Chen 2021). A Decision Transformer is a wrapper around the Transformer architecture introduced in (Vaswani 2017). The general idea is that we use a Transformer to generate trajectories for a Reinforcement Learning problem. More formally, given a trajectory:

$$\tau = (s_1, a_1, r_{1,} s_2, a_2, r_{3,} \dots, s_N, a_N, r_{N,} s_{N+1})$$

The Decisions Transformer uses the underlying Transformer to predict the next action and return in the sequence. We use the action to generate the next state from the environment. Predicted returns are only used during the training phase as part of the loss function. Once we have the action, state, and true return we append these to our context window and feed the new sequence back into the Decision Transformer to generate the next steps in our policy.

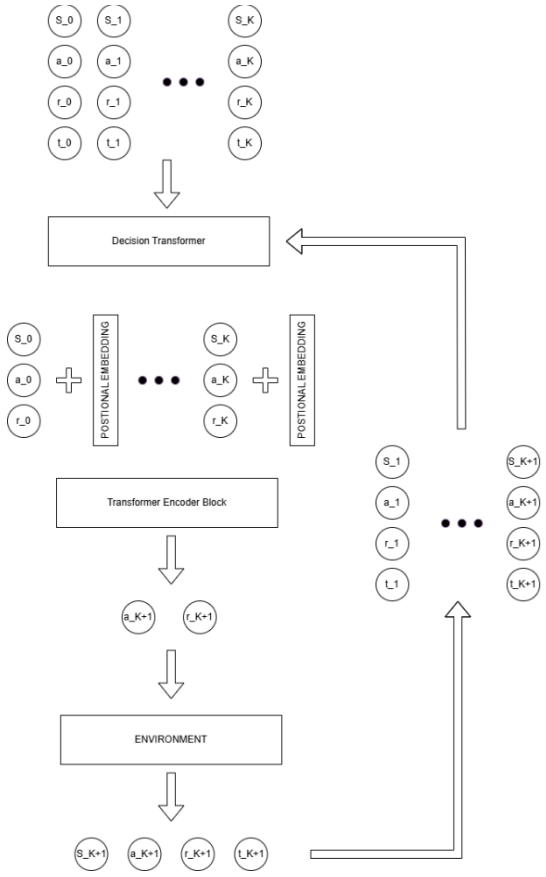

Figure 2 Decision Transformer

As we are restricting ourselves to only those training trajectories generated by human input, we are limited to 25 full training sessions to work with. To stretch this out further, we handled the data as follows:

1. We split the data into “sub-trajectories” each the length of the model’s context size.

2. We used Hindsight Experience Replay (HER) (Andrychowicz 2017) to re-label the rewards on those sub-trajectories. HER was applied to sub-trajectories with 85% probability. We do this in two stages. In stage 1 we extract milestones from the sub-trajectory: Is the ball near the hand? Is the ball near the target? In stage 2, if a milestone was predicted in the previous stage, then we re-label the returns_to_go for that trajectory so that the goal for that run was the sub-goal represented by the milestone we found in stage 1.

We followed the general training procedure described in (Chen 2021), treating the training as a supervised training problem where the goal was to teach the system to predict the correct next action in the sequence. To help the system generalize a bit better, we also asked the system to predict the next reward in the sequence as well, which we rescaled and added to the resulting loss function. Our final loss function can be seen below:

$$loss = MSE(action_{pred} - action_{true}) + 0.001 * MSE(reward_{pred} - reward_{true})$$

Table 1 shows the hyper-parameters we used to train the model with. During training we evaluated the model every 10 epochs. If the maximum reward achieved exceeded the previous maximum reward, we saved a snapshot of the model as a candidate for final evaluation later in the paper. Figure 2 shows the resulting losses and maximum rewards we achieved over the training period.

Hyper-parameter	**Description**	**Value**

Embedding Dimension	Size of the embedding layers used for positional embedding and embedding the state, actions, and rewards.	64
Number of Attention Blocks	How many "Attention" blocks the Transfomer will have.	3
Number of Attention Heads	Number of independent attention heads each Attention Block in the transformer has.	4
Size of Attention Heads	Input size of the Attention Heads.	16
Dropout Rate	Dropout used for the Transformer model	0.3
Context Size	Number of tokens the Transformer will take as input. The maximum length of a trajectory that the system can take in the context is context size / 3 (as each entry in the	30

	sequence requires three tokens)	
Batch Size	Number of sub-trajectories considered for each training loop.	64
Learning Rate		0.0003
Weight Decay	Adds a penalty term proportional to the magnitude of the weights to keep them getting too large.	0.0001
Epochs	Number of training epochs.	200
Evaluation Episodes	Number of episodes to test the model against to determine best reward.	10
Optimizer	Used to perform gradient descent optimizer	AdamW

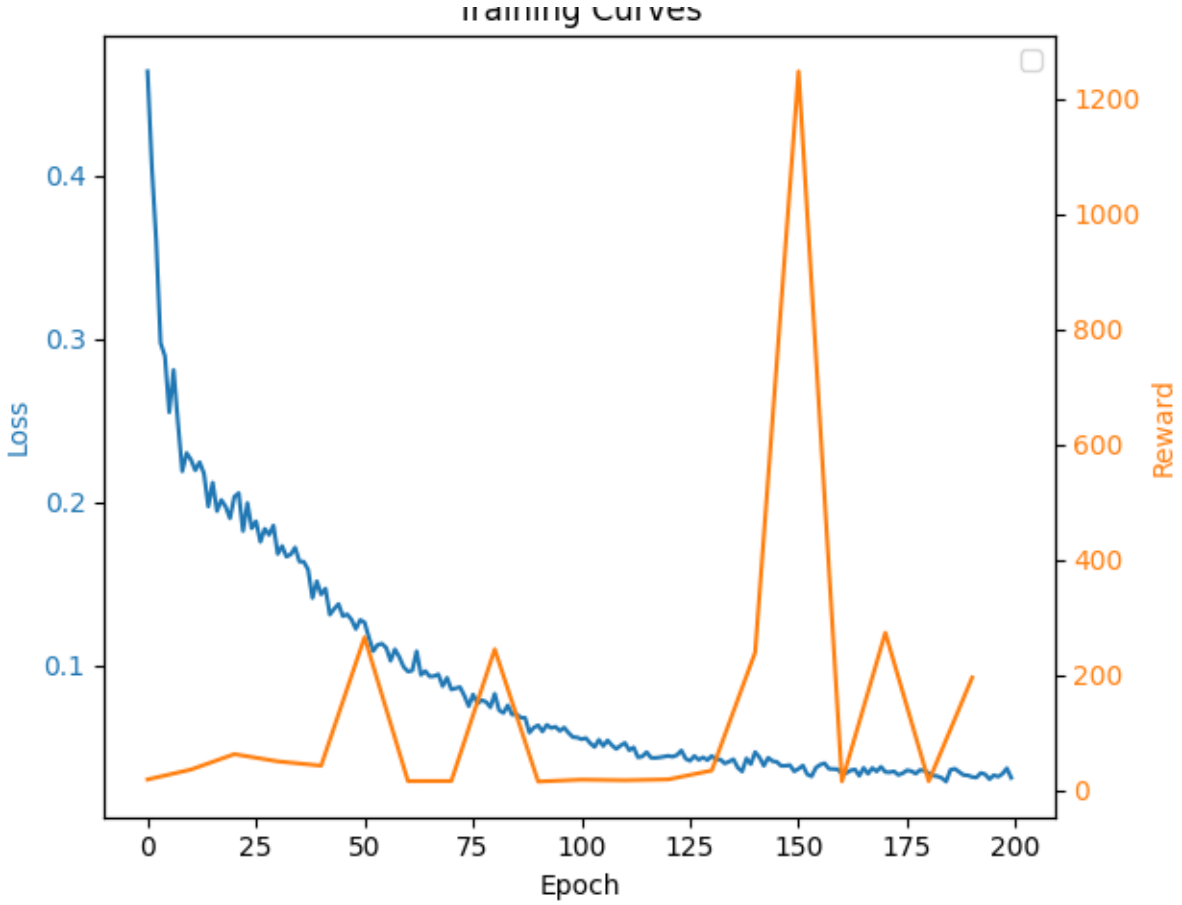

Figure 3 Rewards and Losses from Training

Output Evaluation and Interpretation

We evaluated the model by having it attempt the task ten times and looking for the maximum reward it achieved. Maximum reward was chosen to mitigate the stochastic nature of the task – the ball and target can be anywhere on the board, which could lead the model to have to try situations that are still too difficult for it to generalize to.

After training, the final version of the model we used achieved a maximum reward of 1247.

Outputs from the model are not human readable – the sequence generated is a collection of high-dimensional vectors. Rather than try and interpret these directly, we recorded the model

interacting with the environment and saved the results in mp4 files. These mp4 files have been included with this project's github repo. We recorded four distinct interactions of the model with the environment. Snapshots from each interaction have been collated into figure 4:

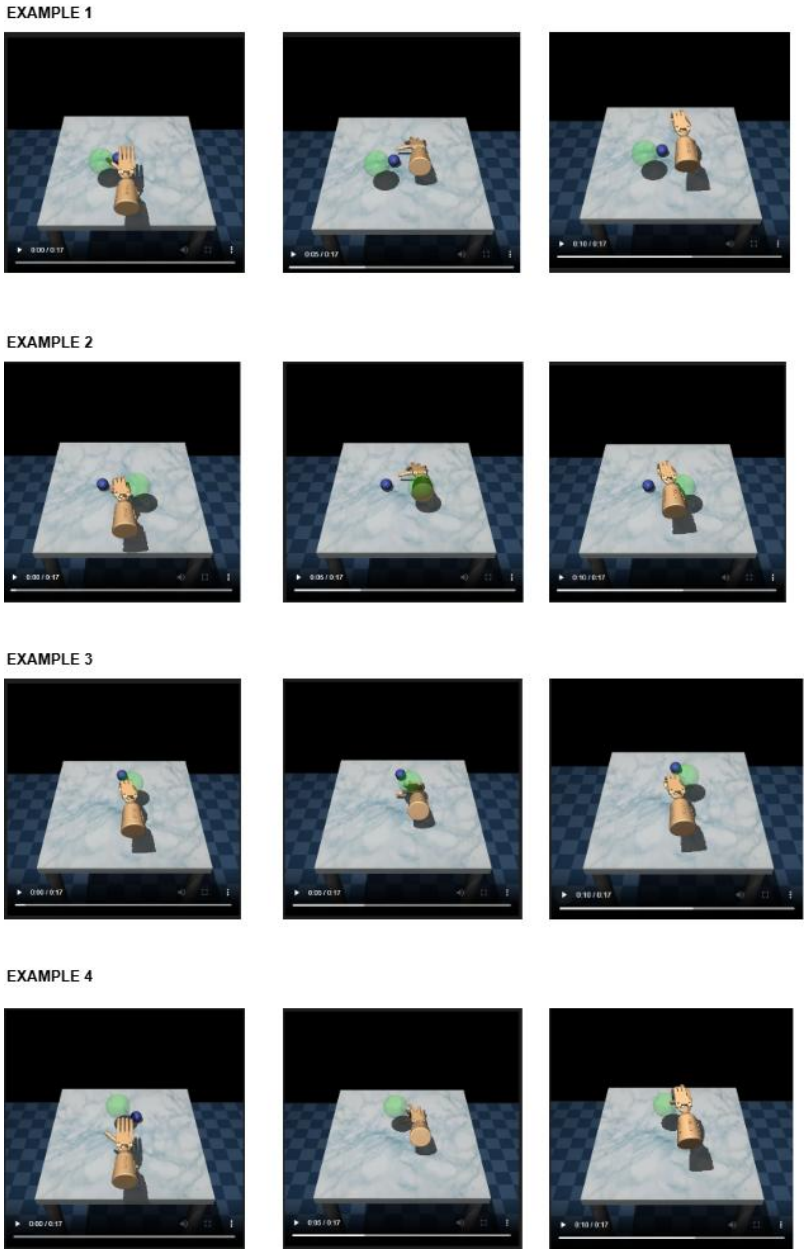

Figure 4 Model interacting with Environment

While the model is still not able to consistently solve the problem, it is able to perform the key actuations that would be needed to pick up the ball itself. The largest issue is that the system is cloning the locational behavior of the example trajectories – i.e. it is performing the correct hand motions to pick up the ball and move it, but in the wrong locations. This is likely caused by the model overfitting the small number of examples we have.

Compared to the jerky and random responses of the untrained model, however, this version is still a significant improvement and may still be useful for some applications. We will discuss how this problem could be mitigated further in the Limitations and Future Improvements section.

Ethical Considerations and Responsible Use

Even with a more diverse training set to work with, there are safety considerations for deploying this system to a real environment. First, this system can only detect the presence of a single object – it has no way to differentiate that object from a human. This means if the system were deployed on a real robot, there would be a risk of harming nearby humans. Also, as we have not modeled the material strength of the ball anywhere, there is also the risk of damaging the very object the model is trained to manipulate.

In its present form, the system has only been trained in a virtual environment. While one could experiment with working with the model directly in a real-world environment, it would be best to recollect the relevant training trajectories in the real world. It's likely that in more complex real-world situations that this system would have a larger number of fail states.

Limitations and Future Improvements

We do not have demographic information on the human subjects used to generate the baseline trajectories this system was trained on. As such, the underlying dataset may not be representative of the full range of human manipulation strategies. It may be the case that people with morphological differences in their hands, missing digits, or suffering from diseases like Parkinson's would handle the ball differently. With such a small dataset, we should not assume that the policies generated are representative of human experience.

As can be seen from the videos (and might be expected from the small number of full trajectories) the model has overfit the data. The primary side effect of this is that while it is performing the correct grasp action (which would be locationally invariant), it tends to perform those actions at the wrong locations.

One future improvement we could make would be to build a hybrid model. The system knows the positional location of the ball (it is provided as part of the observation vector), rather than allow this system to control the full action space, we could limit it to only learn the grasping action, and use a traditional approach to place the palm of the hand over top of the ball.

We could also take a hybrid approach to training the system. We could split the training into two phases: Phase 1 teaches it to model human actions using the algorithm presented above. In phase 2 we would refine the model using a TD-Learning or Monte Carlo approach, allowing the system to learn from own experiences interacting with the environment.

Finally, we could drop the Human generated data only restricted and use the large amount of synthetic data available for solving this problem. Perhaps by combining the human dataset with a synthetically generated expert dataset we can encourage the system to generalize further.

References

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., & Zaremba, W. (2017). *Hindsight experience replay*. arXiv. https://doi.org/10.48550/arXiv.1707.01495

Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., & Mordatch, I. (2021). *Decision transformer: Reinforcement learning via sequence modeling*. arXiv. https://doi.org/10.48550/arXiv.2106.01345

Fu, J., Kumar, A., Nachum, O., Tucker, G., & Levine, S. (2020). *D4RL: Datasets for deep data-driven reinforcement learning*. arXiv. https://doi.org/10.48550/arXiv.2004.07219

Rajeswaran, Aravind, et al. ‘Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations’. CoRR, vol. abs/1709.10087, 2017, http://arxiv.org/abs/1709.10087

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, Illia. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 5998–6008. https://doi.org/10.48550/arXiv.1706.03762